Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1113 Sistemas Empotrados
Prof. Dr.-Ing. Jeferson González Gómez

Taller 4: Yocto Project

1. Proyecto yocto

El Proyecto Yocto es una colaboración de código abierto, enfocada a desarrolladores de Linux embebido, que permite crear imágenes de Linux personalizables para diferentes plataformas de hardware.

El Proyecto Yocto, a través del sistema de construcción OpenEmbedded, proporciona un entorno de desarrollo de código abierto dirigido a las arquitecturas ARM, MIPS, PowerPC y x86, para una variedad de plataformas, incluyendo beaglebone, x86-64, emuladores, entre otras. Se pueden utilizar, además, los componentes de Yocto para diseñar, desarrollar, construir, depurar, emular y verificar las distintas aplicaciones de software usando Linux, Sistema X Window, GNOME orientado a móviles y frameworks basados en Qt.

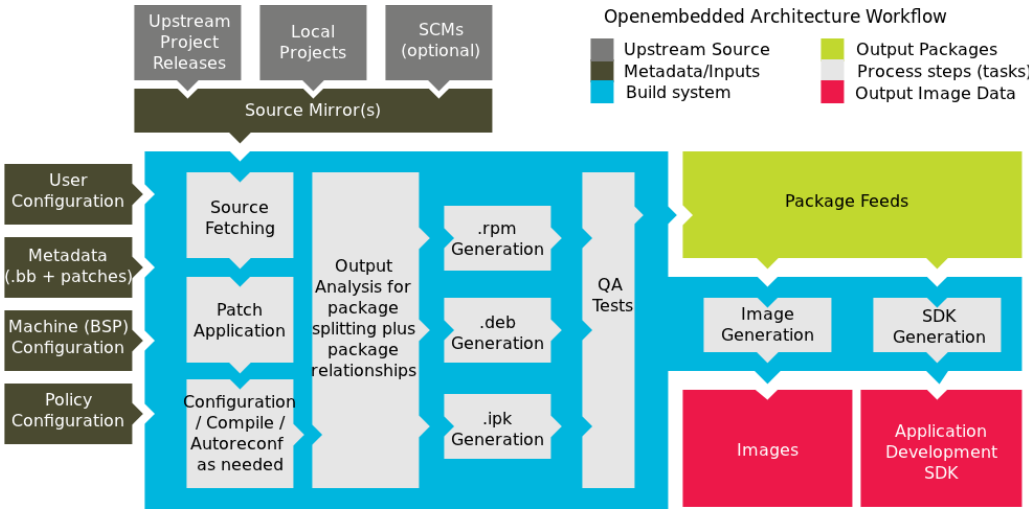


Figura 1: Entorno de desarrollo del Proyecto Yocto

Algunas de las principales características del Proyecto Yocto son:

- Provee un kernel de Linux reciente, junto con un set de comandos y bibliotecas de sistema adecuadas para el entorno embebido.

- Ofrece disponibilidad de componentes de sistema como X11, GTK+, QT, Clutter y SDL (entre otros) lo que permite una experiencia rica en dispositivos con métodos de visualización. Para dispositivos sin esta característica, estos componentes no necesitan ser instalados.
- Crea un núcleo enfocado y estable, compatible con OpenEmbedded, con el cual se puede desarrollar y construir fácil y confiablemente.
- Soporta un amplio rango de hardware y dispositivos de emulación a través de QEMU.

2. Instalando Yocto

Antes de comenzar con la instalación de Yocto, se debe asegurar que se cuenta con el sistema y los paquetes necesarios para su ejecución.

- El sistema anfitrión (*host*) debe tener una versión soportada de Linux. Las distribuciones recomendadas para Scarthgap son:
  Ubuntu **22.04 LTS** o **24.04 LTS** (recomendado), Fedora 40+, openSUSE Leap 15.6+, Debian 12 (Bookworm)+, AlmaLinux 9.3+, Rocky Linux 9.3+.
- Suficiente espacio en disco (hasta 90 GB dependiendo de la imagen y configuración).
- Herramientas base: Git 2.x, tar 1.28+, Python **3.10+** (Python 2 y Python $3 < 3.8$ no son soportados).

Paquetes requeridos

Se debe instalar una lista de paquetes esenciales. En **Ubuntu 22.04/24.04 LTS**:

```
sudo apt install gawk wget git diffstat unzip texinfo gcc build-essential \
    chrpath socat cpio python3 python3-pip python3-pexpect xz-utils \
    debianutils iputils-ping python3-git python3-jinja2 python3-subunit \
    zstd liblz4-tool file locales libacl1
```

Adicionalmente, es necesario configurar el locale del sistema:

```
sudo locale-gen en_US.UTF-8
```

Release Yocto

La versión LTS activa de Yocto es la **5.0 "Scarthgap"** (soporte hasta abril 2028). La versión más reciente disponible se puede consultar en: `https://www.yoctoproject.org/software-overview/downloads/`
Para obtener la última actualización del branch LTS (actualmente **5.0.15**), se recomienda clonar el repositorio oficial con la etiqueta correspondiente:

```
git clone --branch yocto-5.0.15 https://git.yoctoproject.org/poky poky-scarthgap-5.0.15
```

Nota: reemplace `yocto-5.0.15` por el tag más reciente disponible en `https://git.yoctoproject.org/poky/refs/tags`.

3. Creación de imagen para arquitectura emulada

Una vez que se tiene todos los paquetes instalados junto con Yocto, se puede proceder a realizar la primera imagen y emularla por medio de QEMU.

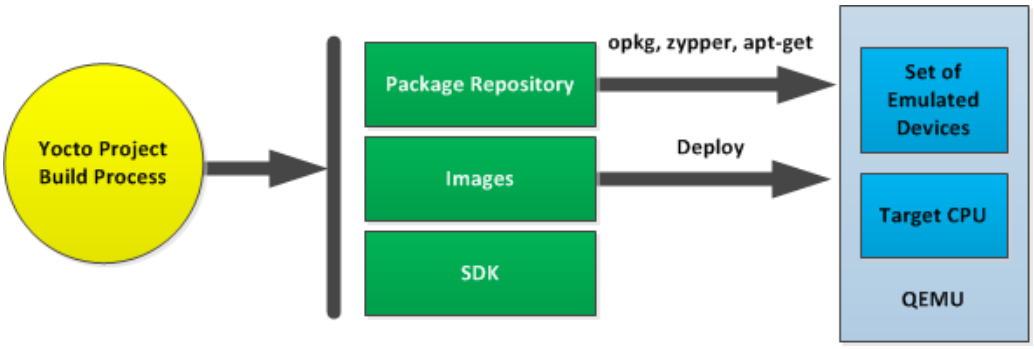


Figura 2: Proceso de construcción de imagen para emulador QEMU

Para realizar esto, se debe seguir los siguientes comandos:

1. Clonar el repositorio de Yocto e ingresar al directorio:

```
git clone --branch yocto-5.0.15 https://git.yoctoproject.org/poky \
    poky-scarthgap-5.0.15
cd poky-scarthgap-5.0.15
```

2. Crear directorio para imagen:

```
mkdir emuARM
```

3. Crear directorio de descargas general (opcional, pero MUY recomendado)

```
mkdir downloads
```

4. Ejecutar script para configuración de ambiente necesario para variables y comandos del proyecto Yocto:

```
source oe-init-build-env emuARM
```

5. Editar archivo de configuración local: (conf/local.conf):

   - Agregar al final: (las rutas deben cambiarse según cada usuario y máquina)

```
DL_DIR ?= "/home/usuario/poky-scarthgap-5.0.15/downloads"
INHERIT += "rm_work"
MACHINE ?= "qemuarm"
```

6. Iniciar construcción de imagen. Puede tardar de 1-4h, dependiendo conexión a internet y capacidad de procesamiento de máquina host (los tiempos han mejorado con las versiones modernas de Yocto):

   ```
   bitbake core-image-minimal
   ```

7. Al finalizar, podrá ejecutar el emulador con la imagen recién creada, ejecutando:

   ```
   runqemu qemuarm
   ```

4. Layers en Yocto

Concepto

Un **layer** (capa) es la unidad de organización modular en Yocto. Cada layer es un directorio que agrupa recetas, configuraciones y parches relacionados con un propósito específico. Esta estructura permite:

- **Reutilización**: un layer desarrollado para un proyecto puede incluirse en otro sin modificación.
- **Separación de responsabilidades**: el **BSP** (*Board Support Package*) del hardware —es decir, el conjunto de drivers, configuración del kernel y archivos de arranque específicos de una placa— va en su propio layer (`meta-bsp`), las aplicaciones de usuario en otro (`meta-myapp`), etc.
- **Mantenibilidad**: los cambios propios nunca tocan los layers oficiales de Poky, lo que facilita actualizar Yocto.

Poky ya incluye varios layers base: `meta`, `meta-poky`, `meta-yocto-bsp`. Los layers activos se declaran en el archivo `conf/bblayers.conf` del directorio de construcción.

Estructura de un layer

```
meta-ejemplo/
  conf/
    layer.conf          <- configuracion del layer
  recipes-ejemplo/
    mipaquete/
      mipaquete_1.0.bb  <- receta
      files/            <- archivos fuente locales (SRC_URI file://)
        ...
```

Crear un layer con bitbake-layers

La forma recomendada de crear un layer es usando la herramienta `bitbake-layers` (disponible tras ejecutar `source oe-init-build-env`):

```
bitbake-layers create-layer ../meta-hello
bitbake-layers add-layer ../meta-hello
```

El primer comando genera la estructura mínima del layer (incluido `conf/layer.conf` y una receta de ejemplo). El segundo lo registra en `conf/bblayers.conf` del directorio de construcción activo.
El archivo `conf/layer.conf` generado tiene esta forma:

```
# We have a conf and classes directory, add to BBPATH
BBPATH .= ":${LAYERDIR}"

# Patrones de archivos de recetas del layer
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "hello"
BBFILE_PATTERN_hello = "^${LAYERDIR}/"
BBFILE_PRIORITY_hello = "6"

LAYERVERSION_hello = "1"
LAYERDEPENDS_hello = "core"
LAYERSERIES_COMPAT_hello = "scarthgap"
```

La variable `LAYERSERIES_COMPAT_hello` declara que este layer es compatible con la versión *scarthgap* de Yocto. Debe actualizarse si se cambia de release.

5. Recetas en Yocto

Concepto

Una **receta** (*recipe*) es un archivo `.bb` que describe cómo obtener, configurar, compilar e instalar un paquete de software dentro del sistema Yocto. BitBake lee estas recetas y ejecuta las tareas correspondientes (`do_fetch`, `do_configure`, `do_compile`, `do_install`, etc.).

Variables fundamentales de una receta

Variable	Descripción
`SUMMARY`	Descripción corta del paquete.
`LICENSE`	Licencia del código fuente.
`LIC_FILES_CHKSUM`	Verificación del archivo de licencia.
`SRC_URI`	Origen del código fuente (git, http, file://, etc.).
`S`	Directorio donde BitBake desempaqueta el código fuente.
`DEPENDS`	Dependencias en tiempo de compilación.
`RDEPENDS`	Dependencias en tiempo de ejecución.
`inherit`	Clase de construcción a heredar (`cmake`, `autotools`, `make`, etc.).

Cuando el código fuente se provee con archivos locales (`file://`), estos deben ubicarse en el subdirectorio `files/` dentro del directorio de la receta.

Ejemplo: receta para *libhello*

Se construirá un layer `meta-hello` con dos recetas: una para la biblioteca *libhello* (de taller03) y otra para la aplicación *hello* que la usa. El código fuente es el mismo presentado en la Sección 1 de taller03.
La estructura completa del layer es:

```
meta-hello/
  conf/
    layer.conf
  recipes-hello/
    libhello/
      libhello_1.0.bb
      files/
        say.c
        say.h
        CMakeLists.txt
    hello/
      hello_1.0.bb
      files/
        main.c
        CMakeLists.txt
```

recipes-hello/libhello/files/CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.10)
project(libhello)

add_library(hello SHARED say.c)
target_include_directories(hello PUBLIC
    $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}>)
```

```
install(TARGETS hello DESTINATION ${CMAKE_INSTALL_LIBDIR})
install(FILES say.h DESTINATION ${CMAKE_INSTALL_INCLUDEDIR})
```

recipes-hello/libhello/libhello_1.0.bb:

```
SUMMARY = "Biblioteca de saludo Hello World"
DESCRIPTION = "Biblioteca compartida con la funcion say_hello()"
LICENSE = "MIT"
LIC_FILES_CHKSUM = \
    "file://${COMMON_LICENSE_DIR}/MIT;md5=0835
       ade698e0bcf8506ecda2f7b4f302"

SRC_URI = "file://say.c \
           file://say.h \
           file://CMakeLists.txt"

S = "${WORKDIR}"

inherit cmake
```

recipes-hello/hello/files/CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.10)
project(hello)

find_package(PkgConfig)
add_executable(hello main.c)
target_link_libraries(hello hello_lib)

install(TARGETS hello DESTINATION ${CMAKE_INSTALL_BINDIR})
```

recipes-hello/hello/hello_1.0.bb:

```
SUMMARY = "Aplicacion Hello World usando libhello"
LICENSE = "MIT"
LIC_FILES_CHKSUM = \
    "file://${COMMON_LICENSE_DIR}/MIT;md5=0835
       ade698e0bcf8506ecda2f7b4f302"

DEPENDS = "libhello"

SRC_URI = "file://main.c \
           file://CMakeLists.txt"

S = "${WORKDIR}"

inherit cmake

# Dependencia en tiempo de ejecucion: libhello debe estar en la imagen
RDEPENDS:${PN} = "libhello"
```

Agregar los paquetes a la imagen

Una vez creado el layer y las recetas, se deben incluir los paquetes en la imagen editando `conf/local.conf` dentro del directorio de construcción:

```
IMAGE_INSTALL:append = " hello libhello"
```

Luego se reconstruye la imagen:

```
bitbake core-image-minimal
```

Al iniciar QEMU con `runqemu qemuarm` y ejecutar `hello` en el terminal del sistema embebido, se obtendrá la salida `Hello, World!`.

6. Evaluación

Descripción

Utilizando el entorno Yocto configurado en este taller, debe crear un layer propio `meta-operaciones` que contenga recetas para construir e instalar en la imagen QEMU la biblioteca *liboperaciones* y su aplicación *calculadora* (descritas en el Taller 3).
Los requisitos son:

- **Layer meta-operaciones**: creado con `bitbake-layers create-layer`, con `LAYERSERIES_COMPAT` configurado para *scarthgap*.
- **Receta liboperaciones**: biblioteca compartida con las cinco funciones matemáticas (suma, resta, multiplicación, división, raíz cuadrada). Debe usar la clase `cmake` y proveer los archivos fuente vía `file://`.
- **Receta calculadora**: aplicación que llama a cada función e imprime los resultados. Debe declarar `DEPENDS` y `RDEPENDS` sobre *liboperaciones*.
- **Imagen**: ambos paquetes incluidos en `core-image-minimal` via `IMAGE_INSTALL:append`.
- **Verificación en QEMU**: el binario `calculadora` debe ejecutarse correctamente dentro del emulador y mostrar los resultados de las cinco operaciones.

Entregables

(subir al TecDigital)

- Archivo `.tar` del directorio `meta-operaciones` completo (incluya su nombre en el nombre del archivo).
- Documento PDF con capturas de pantalla que muestren:
  1. La salida de `uname -a` dentro de QEMU.
  2. La salida del comando `calculadora` dentro de QEMU, con los resultados de las cinco operaciones matemáticas.